

第 1 章

引言：一些典型问题

1.1 第一个问题：稳定匹配

作为开场话题，我们来看一个算法问题，它很好地说明了我们将要强调的许多主题。它源自一些非常自然和实际的问题，从这些问题中，我们形式化了一个简洁明了的问题陈述。解决问题的算法也非常简洁，我们的大部分工作在于证明它是正确的，并对算法终止、得到答案所花费的时间量给出可以接受的界。稳定匹配问题（Stable Matching Problem）本身有几个起源。

问题

稳定匹配问题的一部分起源于 1962 年。当时，两位数学经济学家 David Gale 和 Lloyd Shapley 提出了这样一个问题：是否可以设计一个本身具有强制力的（self-enforcing）大学录取过程或招聘过程？他们这么说是什么意思呢？

为了明白这个问题，我们先非正式地想想一群朋友可能出现的情况，他们都是大学计算机专业的大三学生，开始申请暑期实习公司。申请过程的关键是两个不同类型的群体之间的相互作用：公司（雇主）和学生（申请人）。每个申请人都有对公司的偏好顺序，每个公司（一旦收到申请）对其申请人会有偏好顺序。根据这些偏好，公司向一些申请人发出录用通知，申请人选择接受哪个录用，开始他们的暑期实习。

如果没有任何强制现状的机制，Gale 和 Shapley 考虑了这个过程可能出现问题的各种情况。例如，假设你的朋友 Raj 刚刚接受了大型电信公司 CluNet 的暑期工作。几天后，一直拖着没有做出最终决定的小型初创公司 WebExodus 打电话给 Raj，为他提供暑期工作。现在，Raj 实际上更想去 WebExodus 而不是 CluNet（也许是因为环境宽松，任何事都可能发生的氛围），因此，这个新情况很可能让他反悔接受 CluNet 的录用，改去 WebExodus。突然少了一名暑期实习生，CluNet 为申请人等待列表中的下一位提供了这份工作，他迅速反悔了之前接受的软件巨头 Babelsoft 的录用，情况开始失控。

从另一方来看，事情同样糟糕，甚至更糟。假设 Raj 的朋友 Chelsea 决定要去 Babelsoft，但刚刚听到了 Raj 的故事，她打电话给 WebExodus 的人说：“你知道，我真的宁愿和你们一起度过夏天，而不是去 Babelsoft。”他们发现这很容易令人相信。此外，在查看 Chelsea 的申请时，他们意识到他们宁愿雇用她，而不是其他一些实际上已安排好在 WebExodus 度过夏天的学生。在这种情况下，如果 WebExodus 是一个稍微不那么谨慎的公司，那么它可能会找到一些方法来撤回对其他学生的录用，并雇用 Chelsea。

像这样的情况很快会产生很多混乱，很多人（无论是申请人还是雇主）最终都会对这个过程和结果感到不满。出了什么问题？一个基本问题是，这个过程不是本身具有强制力的：如果

允许人们凭自己的利益行事，那么它就有可能被毁掉。

我们可能更喜欢以下更稳定的情况，其中自身利益本身阻止了录用的撤回和改发。考虑另一个学生，他已经安排在 CluNet 度过夏天，但打电话给 WebExodus 表示他宁愿为他们工作。但在这种情况下，根据已经接受的录用通知，他们能回答：“不，实际上我们更喜欢我们已经接受的每个学生，所以恐怕我们帮不上忙。”或者考虑一下雇主，认真跟进那些去了其他地方的顶级申请人。结果每个人都告诉他：“不，我在现在的地方很开心。”在这种情况下，所有的结果都是稳定的：不会发生进一步的幕后交易。

这就是 Gale 和 Shapley 提出的问题：鉴于雇主和申请人的一系列偏好，我们是否可以将申请人分配给雇主，以便对于每个雇主 E 和每个未安排为 E 工作的申请人 A ，以下两种情况至少一种成立。

(i) E 更偏好他接受的每一个申请人，而不是 A 。

(ii) A 更偏好她目前的状况，而不是为雇主 E 工作。

如果这样，结果就是稳定的：个人自身利益将阻止所有申请人/雇主的幕后交易。

Gale 和 Shapley 开始为这个问题开发一个引人注目的算法解，我们马上就讨论这个问题。在此之前，注意，这不是稳定匹配问题的唯一起源。事实表明，在 Gale 和 Shapley 开始这项工作之前的 10 年，他们并不知道，美国国家医生匹配计划（National Resident Matching Program）一直采用一种非常相似的程序，具有相同的潜在动机，让医生与医院匹配。实际上，这个系统变化相对较小，至今仍在使用。

这是该问题具有根本吸引力的一个证明。从本书的角度来看，它为我们提供了第一个很好的领域，可以推出一些基本的组合定义和基于它们的算法。

问题形式化

理解这个概念的本质，有助于让问题尽可能清晰。公司和申请人的世界包含一些分散注意力的不对称性。每个申请人都在寻找一家公司，但每家公司都在寻找多名申请人。此外，申请人可能比暑期工作的职位更多（或在某些时候更少）。最后，每个申请人通常不会向每家公司提出申请。

至少在初始阶段，消除这些复杂性并得出问题的更“简单”版本，是有用的： n 个申请人都向 n 家公司提出申请，并且每个公司都想接受单个申请人。我们会看到，这样做可以保留该问题内含的基本问题：特别是，我们对这个简化版本的解也将直接扩展到更一般的情况。

顺着 Gale 和 Shapley 的思路，我们发现这个特殊情况可以看成是设计一个系统的问题，通过这个系统， n 位男士和 n 位女士中的每个人最终都可以结婚：我们的问题自然类似于两种“性别”（申请人和公司），在我们考虑的情况下，每个人都在寻求与一名异性配对^①。

因此，考虑 n 位男士的集合 $M = \{m_1, \dots, m_n\}$ ，以及 n 位女士的集合 $W = \{w_1, \dots, w_n\}$ 。令 $M \times W$ 表示形如 (m, w) 的所有可能有序对的集合，其中 $m \in M$ 且 $w \in W$ 。匹配 S 是一组有序对，每个来自 $M \times W$ ，并具有以下属性： M 的每个成员和 W 的每个成员在 S 中最多出现在一对中。完美匹配（perfect matching） S' 是具有这种属性的匹配： M 的每个成员和 W 的每个成员在 S' 中恰好出现在一对中。

匹配和完美匹配是整本书中经常出现的术语。它们在模拟各种算法问题时自然而然地出现。在目前的情况下，一个完美匹配只对应于一种方式将男士与女士配对，每个人最终都嫁给了某个人，没有人与多人结婚：既没有单身，也没有多配偶制。

^① Gale 和 Shapley 也考虑了同性稳定匹配问题，其中只有—个性别。这是由相关应用推动的，但事实证明它在技术层面上相当不同。鉴于这里考虑的申请人——雇主应用，我们将专注于具有两种性别的版本。

现在我们可以加入偏好的概念。每位男士 $m \in M$ 对所有女士进行排名。如果 m 对 w 的排名高于 w' ，我们会说 m 更喜欢 w ，而不是 w' 。我们将 m 的有序排名称为他的偏好列表。排名中不允许出现并列。类似地，每位女士都对所有男士进行排名。

给定一个完美匹配 S ，会出现什么问题？根据在雇主和申请人方面的最初动机，我们应该担心以下情况： S 中有两对，即 (m, w) 和 (m', w') （如图 1-1 所示），具有这样的性质： m 更喜欢 w' 而不是 w ， w' 更喜欢 m 而不是 m' 。在这种情况下，没有什么可以阻止 m 和 w' 放弃现有的伴侣，走到一起。这组婚姻不是本身具有强制力的。我们会说，对于 S ，这样一对 (m, w') 是不稳定因素（instability）： (m, w') 不属于 S ，但 m 和 w' 更喜欢彼此，而不是 S 中他们的伴侣。

那么，我们的目标就是一组没有不稳定因素的婚姻。如果 (i) 它是完美的，并且 (ii) 对于 S 没有不稳定因素，我们就说匹配 S 是稳定的。下面两个问题立即浮现在脑海中。

- 对于所有偏好列表，是否存在稳定匹配？
- 给定一组偏好列表，如果存在稳定匹配，我们是否可以有效地构造？

一些例子

为了说明这些定义，请考虑以下两个非常简单的稳定匹配问题实例。

第一个例子，假设有一组两位男士 $\{m, m'\}$ 和一组两位女士 $\{w, w'\}$ 。偏好列表如下：

m 更喜欢 w ，而不是 w' ；
 m' 更喜欢 w ，而不是 w' ；
 w 更喜欢 m ，而不是 m' ；
 w' 更喜欢 m ，而不是 m' 。

如果我们直观地考虑这组偏好列表，它展示了完全一致性：男士同意女士的顺序，女士同意男士的顺序。这里有一个独特的稳定匹配，由 (m, w) 和 (m', w') 配对组成。由 (m', w) 和 (m, w') 配对组成的另一个完美匹配不是稳定匹配，因为对于该匹配， (m, w) 配对将构成不稳定因素。（ m 和 w 都希望离开他们各自的伴侣并配对。）

下一个例子，情况有点复杂。假设偏好是：

m 更喜欢 w ，而不是 w' ；
 m' 更喜欢 w' ，而不是 w ；
 w 更喜欢 m' ，而不是 m ；
 w' 更喜欢 m ，而不是 m' 。

这种情况下会发生什么？这两位男士的偏好完全相互配合（他们将不同的女士排在首位），两位女士的偏好同样完美地相互配合。但男士的偏好与女士的偏好完全冲突。

在第二个例子中，有两种不同的稳定匹配。由 (m, w) 和 (m', w') 配对组成的匹配是稳定的，因为两位男士都最开心，所以两人都不会离开他们配对的伴侣。但由 (m', w) 和 (m, w') 配对组成的匹配也是稳定的，理由刚好相反，因为两位女士都最开心。这是我们下面要记住的重要一点：实例可能有多个稳定匹配。

不稳定因素： m 和 w' 更喜欢别人，而不是他们当前的伴侣。

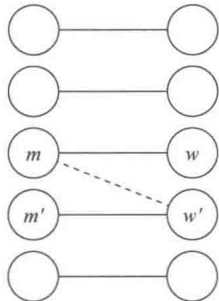


图 1-1 具有不稳定因素 (m, w') 的完美匹配 S

设计算法

我们现在证明，男女之间的每组偏好列表都存在稳定的匹配。而且，我们证明这一点的方

法也会回答上面提到的第二个问题：我们将提供一个有效的算法，它以一些偏好列表为输入，构造一个稳定匹配。

让我们来考虑推动该算法的一些基本思想。

- 最初，每个人都未婚。假设一个未婚男士 m 选择在他的偏好列表中排名最高的女士 w ，并向她求婚。我们能否立即声称 (m, w) 将成为最终稳定匹配中的一对？不一定：在未来的某个时刻， w 更喜欢的男士 m' 可能向她求婚。另外， w 立即拒绝 m 也是危险的。她可能永远不会收到她的排名中像 m 这么高的人求婚。因此，一个自然的想法是让 (m, w) 配对进入中间状态：订婚。
- 假设我们现在处于某种状态，其中一些男士和女士是自由的（没有订婚），有些人已订婚。下一步可能是这样的。任意一个自由男士 m 选择了他尚未求婚的排名最高的女士 w ，向她求婚。如果 w 也是自由的，那么 m 和 w 就会订婚。否则， w 已经与另一位男士 m' 订婚。在这种情况下，她决定 m 和 m' 中的哪一个在她的偏好列表中排名更高，这个人与 w 订婚，另一个人变为自由。
- 最后，算法将在没有人自由时终止。此时，所有订婚都被宣布为最终结果，并返回得到的完美匹配。

下面是 Gale-Shapley 算法（简称 G-S 算法）的具体描述。

最初所有 $m \in M$ 和 $w \in W$ 都是自由的

While 存在一位男士 m ，他是自由的，而且还没有向每位女士求过婚

 选择这样一位男士 m

 设 w 是 m 的偏好列表中排名最高的女士，而 m 还没有向她求过婚

 If w 是自由的 then

(m, w) 变成订婚状态

 Else w 当前与 m' 订婚

 If w 更喜欢 m' 而不是 m then

m 保持自由

 Else w 更喜欢 m 而不是 m'

(m, w) 变成订婚状态

m' 变成自由的

 Endif

Endif

Endwhile

Return 订婚配对的集合 S

如果女士 w 更喜欢 m 而不是 m' ，她会和 m 订婚。

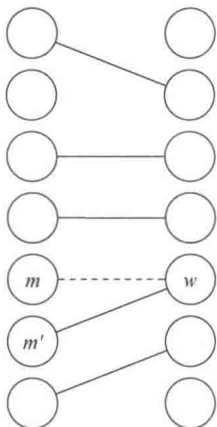


图 1-2 当自由男士 m 向女士 w 求婚时，G-S 算法的中间状态

图 1-2 描述了算法的一个状态。

一个有趣的事情是，尽管 G-S 算法很容易陈述，但并不是很明显它会返回稳定匹配，甚至是完美匹配。我们现在通过一系列中间事实来证明这一点。

分析算法

首先考虑在执行算法期间女士 w 的视角。有一段时间，没有人向她求婚，而且她是自由的。然后一位男士 m 可能向她求婚，接着她就会订婚。随着时间的推移，她可能会收到另外的求婚，接受那些提高伴侣排名的求婚。因此，我们发现以下事实：

(1.1) w 从她收到第一次求婚的那一刻开始就保持订婚状态，并且她所订婚的伴侣的排名越来越好（就她的偏好列表而言）。

在执行算法期间，男士 m 的视角是相当不同的。在他向名单上排名最高的女士求婚之前，他是自由的；经过这次求婚，他可能会订婚也可能不会订婚。随着时间的推移，他可以在自由和订婚之间交替；但是，确实会保持以下性质：

(1.2) m 求婚的女士排名越来越差（根据他的偏好列表）。

现在我们证明该算法会终止，并给出终止所需的最大迭代次数的界。

(1.3) G-S 算法在 While 循环中最多迭代 n^2 次之后终止。

证 要确定算法运行时间上界，一种有效的策略就是找到一个进展（progress）的度量。我们在这里正要尝试这么做。也就是说，我们寻求一些精确的说法，表明算法采取的每个步骤都使其更接近终止。

在本算法的情况下，每次迭代都包含某位男士（仅一次）向他从未求过婚的女士求婚。因此，如果令 $\mathcal{P}(t)$ 表示 (m, w) 对的集合，其中 m 在迭代 t 结束时向 w 求过婚，我们会看到，对于所有 t ， $\mathcal{P}(t+1)$ 的大小严格大于 $\mathcal{P}(t)$ 的大小。但是，由于总共只有 n^2 种可能的男士和女士配对，因此在算法过程中 $\mathcal{P}(\cdot)$ 的值最多会增加 n^2 次。因此，最多会进行 n^2 次迭代。■

关于前一事实及其证明，有两点值得注意。首先，算法的执行（对于某些偏好列表）可能包含接近 n^2 次迭代，因此这种分析与最佳可能结果相差不大。其次，有许多量不能很好地作为算法的进展度量（progress measure），因为它们不一定在每次迭代中严格增加。例如，自由个人的数量可以从一次迭代到下一次迭代保持不变，订婚对的数量也是如此。因此，这些量不能直接用于给出最大可能迭代次数的上限，像上一段那样的方式。

现在让我们证明，在算法终止时返回的集合 S 实际上是完美匹配。为什么这不是很明显？从本质上讲，我们必须证明，没有男士会在他的偏好列表结束时仍然“落单”；While 循环退出的唯一方法是没有自由男士。在这种情况下，订婚夫妇的集合确实是完美匹配。

因此，我们需要证明的主要内容如下：

(1.4) 如果 m 在执行算法的某个时刻是自由的，就有一个他尚未求婚的女士。

证 假设 m 是自由的，但已经向每位女士求过婚。那么根据 (1.1)， n 位女士中的每一位都在这个时间点已订婚。因为已订婚对的集合形成一个匹配，所以此时也必定有 n 位订婚男士。但是，总共只有 n 位男士，并且 m 没有订婚，所以这是一个矛盾。■

(1.5) 终止时返回的集合 S 是完美匹配。

证 订婚对的集合总是形成一个匹配。让我们假设算法终止时有一个自由男士 m 。在终止时， m 肯定已经向每位女士求过婚，否则 While 循环将不会退出。但这与 (1.4) 相矛盾，它表明不可能有一位男士向每位女士求过婚还仍然自由。■

最后，我们证明算法的主要特性，即它得到一个稳定匹配。

(1.6) 考虑 G-S 算法的一次执行，它返回一个配对集合 S 。集合 S 是稳定匹配。

证 我们已经在 (1.5) 中看到 S 是完美匹配。因此，为了证明 S 是稳定匹配，我们将假设存在关于 S 的不稳定因素并导致矛盾。如前所述，这种不稳定因素将涉及 S 中的两个配对，即 (m, w) 和 (m', w') ，它们具有以下属性：

- m 更喜欢 w' ，而不是 w ，并且

- w' 更喜欢 m ，而不是 m' 。

在执行生成 S 的算法时，根据定义， m 的最后一次求婚是 w 。现在我们问：在这次执行的早些时候， m 是否向 w' 求过婚？如果没有向 w' 求过婚，那么 w 肯定在 m 的偏好列表上排名比 w' 更高，这与我们假设“ m 更喜欢 w' ，而不是 w ”相矛盾。如果他向 w' 求过婚，那么他被 w 拒绝了，因为 w' 更喜欢另一位男士 m'' ，而不是 m 。 m' 是 w' 的最终伴侣，因此，要么 $m'' = m'$ ，要么根据 (1.1)， w' 更喜欢她最后的伴侣 m' ，而不是 m'' 。这两种情况都与我们的假设“ w' 更喜欢 m ，而不是 m'' ”相矛盾。

由此得出 S 是稳定匹配。■

扩展

从定义稳定匹配的概念开始，我们刚刚证明了 G-S 算法实际上构造了一个稳定匹配。现在进一步考虑一些问题，也就是关于 G-S 算法的行为，以及它与不同稳定匹配的性质关系。

首先，回想一下之前看到的一个例子，其中可能存在多个稳定匹配。复述一下，例子中的偏好列表如下：

m 更喜欢 w ，而不是 w' ；
 m' 更喜欢 w' ，而不是 w ；
 w 更喜欢 m' ，而不是 m ；
 w' 更喜欢 m ，而不是 m' 。

现在，在 G-S 算法的任何一次执行中， m 都会与 w 订婚， m' 都会与 w' 订婚（或许顺序不同），情况就会保持在那里。因此，由男士求婚的 G-S 算法执行不能获得由配对 (m', w) 和 (m, w') 构成的另一个稳定匹配。然而，如果我们运行女士求婚的算法版本，就会得到。在更大的例子中，双方都超过两个人，甚至会有更多可能的稳定匹配，其中许多是无法通过任何自然算法得到的。

这个例子显示了 G-S 算法中的某种“不公平”，它有利于男士。如果男士的偏好完全相互配合（他们都将不同的女士列为首选），那么在 G-S 算法的所有运行中，所有男士最终都会与他们的第一选择相匹配，而不依赖于女士的偏好。如果女士的偏好与男士的偏好完全冲突（就像本例中的情况那样），那么得到的稳定匹配对女士就最差。因此，这个简单的偏好列表简要地总结了某人注定会最终不开心的情境：如果男士求婚，女士会不开心；如果女士求婚，男士会不开心。

现在让我们更详细地分析 G-S 算法，并尝试理解这种“不公平”现象的普遍性。

首先，我们的例子支持这样的观点：G-S 算法实际上是部分指定的——只要有一个自由男士，我们就可以选择任何自由男士来进行下一次求婚。不同的选择指定了算法的不同执行。这就是为什么，要小心，我们在 (1.6) 中说“考虑 G-S 算法的一次执行，它返回一个配对集合 S ”，而不是说“考虑 G-S 算法返回的集合 S ”。

因此，我们遇到另一个非常自然的问题：G-S 算法的所有执行都会产生相同的匹配吗？这是计算机科学中许多背景下出现的一个问题类型：有一个异步（asynchronously）运行的算法，不同的独立部分执行的动作可以以复杂方式交错，我们想知道，这种异步对最终结果导致的变化有多大。考虑一个非常不同的例子，独立部分可能不是男士和女士，而是控制飞机机翼各部件的电子元件。异步对它们行为的影响可能是一个大问题。

在目前的上下文中，我们会看到问题的答案非常简洁：G-S 算法的所有执行产生相同的匹配。我们现在继续证明这一点。

所有执行产生相同的匹配

有许多可能的方法来证明这一点，其中许多会导致相当复杂的论证。事实表明，对我们来说最简单、最有信息的方法是对所获得的匹配给出唯一的特征（characterize），然后证明所有执行都会得到具有此特征的匹配。

这个特征是什么？我们将证明，每位男士最终将得到具体意义上的“最佳可能伴侣”。（回想一下，如果所有男士都喜欢不同的女士，这是真的。）首先，如果有一个稳定匹配包含配对 (m, w) ，我们会说女士 w 是男士 m 的有效伴侣（valid partner）。如果 w 是 m 的有效伴侣，并且在 m 的排名中，没有高于 w 的女士是他的有效伴侣，我们会说 w 是 m 的最佳有效伴侣（best valid partner）。我们用 $\text{best}(m)$ 来表示 m 的最佳有效伴侣。

现在，令 S^* 表示配对的集合 $\{(m, \text{best}(m)) : m \in M\}$ 。我们将证明以下事实。

(1.7) 每次执行 G-S 算法都会产生集合 S^* 。

这种说法在许多层面都令人惊讶。首先，按照定义，没有理由相信 S^* 是匹配，更不用说是稳定匹配。毕竟，为什么两位男士不会拥有同样的最佳有效伴侣呢？其次，结果表明 G-S 算法同时为每个人提供了最好的结果。不存在一个稳定匹配，其中任何一位男士有希望得到更好的结果。最后，它证明了 G-S 算法中的求婚顺序对最终结果完全没有影响，从而回答我们上面的问题。

尽管如此，证明并不那么难。

证 利用反证法，让我们假设，G-S 算法的某些执行 E 导致匹配 S ，其中某位男士配对的女士不是他的最佳有效伴侣。由于男士按偏好的降序来求婚，因此这意味着在算法的执行 E 期间，某位男士被有效伴侣拒绝。因此，考虑执行 E 期间的第一个时刻，其中某位男士，如 m ，被有效伴侣 w 拒绝。同样，由于男士按偏好的降序来求婚，并且由于这是第一次发生这样的拒绝，因此肯定 w 是 m 的最佳有效伴侣 $\text{best}(m)$ 。

m 被 w 拒绝，要么是因为 m 求婚而 w 更喜欢已有的订婚对象，要么是因为 w 放弃了她与 m 的订婚而选择了更好的求婚。但不管怎样，在这一刻， w 进入或继续保持与男士 m' 的订婚状态，她更喜欢 m' 而不是 m 。

由于 w 是 m 的有效伙伴，因此存在一个稳定匹配 S' ，其中包含配对 (m, w) 。现在我们问：在这个匹配中谁与 m' 配对？假设是一位女士 $w' \neq w$ 。

由于执行 E 中， m 被 w 拒绝是有效伴侣对男士的第一次拒绝，因此，当 m' 与 w 订婚时， m' 肯定没有被任何有效伴侣拒绝过。因为他按偏好的降序求婚，并且因为 w' 显然是 m' 的有效伙伴，肯定 m' 更喜欢 w 而不是 w' 。但是，我们已经看到 w 更喜欢 m' 而不是 m ，因为在执行 E 中，她拒绝了 m 而选择了 m' 。由于 $(m', w) \notin S'$ ，因此 (m', w) 是 S' 中的不稳定因素。

这与我们声称 S' 是稳定的相矛盾，因此与最初的假设相矛盾。■

因此，对男士来说，G-S 算法是理想的。不幸的是，对于女士就不能这样说了。对于女士 w ，如果存在包含该对 (m, w) 的稳定匹配，我们就说 m 是有效的伴侣。如果 m 是 w 的有效伴侣，并且没有一个排名低于 m 的人是她的有效伴侣，我们就说 m 是 w 的最差有效伴侣（worst valid partner）。

(1.8) 在稳定匹配 S^* 中，每位女士都与她的最差有效伴侣配对。

证 假设在 S^* 中存在一对 (m, w) ，其中 m 不是 w 的最差有效伴侣。然后存在一个稳定匹配 S' ，其中 w 与一位男士 m' 配对，她对 m' 的喜欢程度还不及 m 。在 S' 中，与 m 配对的女士 $w' \neq w$ 。

因为 w 是 m 的最佳有效伴侣，而 w' 是 m 的有效伴侣，我们看到 m 更喜欢 w 而不是 w' 。

但由此可见， (m, w) 是 S' 的不稳定因素，与声称 S' 是稳定的相矛盾，因此与我们最初的假设相矛盾。■

因此我们发现，在上面的简单例子中，男士的偏好与女士的冲突，这暗示了一个非常普遍的现象：对于任何输入，在 G-S 算法中求婚的一方最终会得到最好的稳定匹配（从他们的角度来看），而不求婚的一方最终会得到最差稳定匹配。

1.2 5 个典型问题

稳定匹配问题为我们提供了算法设计过程的丰富示例。对于许多问题，这个过程涉及一些重要的步骤：用足够的数学精度来形式化问题，让我们可以提出一个具体的问题，并开始考虑解决它的算法；设计问题的算法；证明算法是正确的并给出运行时间的界，以此来分析算法，从而确定算法的效率。

这种高级策略在实践中得益于一些基本设计技术的帮助，这些技术在评估问题的固有复杂性和制定算法来解决问题时非常有用。与任何领域一样，熟悉这些设计技术是一个渐进的过程；但是根据经验，人们可以开始认识到问题属于可识别的类型，并认识到问题陈述中的微妙变化如何对其计算难度产生巨大影响。

为了开始讨论，挑出我们在算法研究中遇到的几个典型的里程碑式的问题是有帮助的。它们是一些清晰形式化的问题，在一般层面上彼此相似，但在难度和人们对它们采取的各种方法上迥然不同。前三个问题将通过一系列越来越精妙的算法技术有效地解决；第四个问题标志着我们讨论中的一个重要转折点，作为一个问题的例子，人们相信没有任何有效算法可以解决它；第五个问题暗示着一类问题，人们相信这更困难。

这些问题是独立的，并且都是由计算应用推动的。但是，要讨论其中一些问题，使用图（graph）的术语是有帮助的。虽然图是前期计算机科学课程中的常见主题，但我们将在第 3 章中以相当深入的方式介绍它们。由于它们强大的表现力，因此会在整本书中广泛使用它们。对于这里的讨论，只要将图 G 视为在一组对象之间编码配对关系的简单方法就足够了。因此， G 由一对集合 (V, E) 构成：节点集合 V 和边（edge）的集合 E ，每条边“连接”两个节点。于是，我们将边 $e \in E$ 表示为 V 的两元素子集： $e = \{u, v\}$ ， $u, v \in V$ ，其中我们将 u 和 v 称为 e 的端点。我们通常绘制如图 1-3 所示的图形，每个节点画成一个小圆圈，每条边画成连接其两端点的线段。

现在让我们来讨论 5 个典型问题。

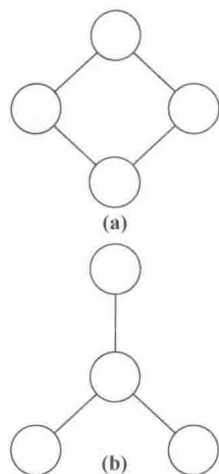


图 1-3 (a) 和 (b) 都描绘了一个四节点的图

区间调度

考虑以下非常简单的调度问题。你有一个资源（可能是一个演讲室、一台超级计算机或一台电子显微镜），许多人要求在一段时间内使用这些资源。请求（request）采取以下形式：我可以从时间 s 开始直到时间 f 为止，预定这个资源吗？我们假设资源一次最多只能由一个人使用。调度程序希望接受这些请求的子集，拒绝所有其他请求，以便接受的请求不会在时间上重叠。目标是让接受的请求数最大。

更形式地说, 有 n 个请求, 标记为 $1, \dots, n$, 每个请求 i 指定开始时间 s_i 和结束时间 f_i 。当然, 对于所有 i , 都有 $s_i < f_i$ 。如果两个请求 i 和 j 的区间不重叠, 则它们是相容的 (compatible), 即请求 i 的时间区间早于请求 j ($f_i \leq s_j$), 或者请求 i 的时间晚于请求 j ($f_j \leq s_i$)。更一般地说, 对于请求的子集 A , 如果每对请求 $i, j \in A, i \neq j$ 是相容的, 那么该子集是相容的。目标是选择一个最大的相容请求子集。

图 1-4 展示了这个区间调度问题的一个实例。注意, 有一个大小为 4 的相容集合, 这是最大的相容集合。

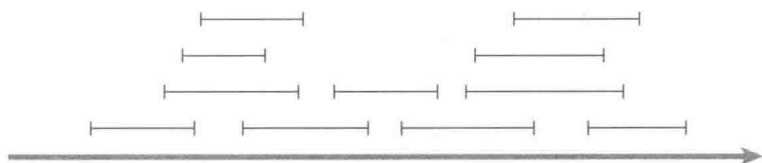


图 1-4 区间调度问题的一个实例

我们很快就会看到, 这个问题可以用一种非常自然的算法来解决, 该算法根据某种启发式方法对请求集进行排序, 然后“贪心地”对它们处理一遍, 选择尽可能大的相容子集。这是一类典型的贪心算法 (greedy algorithm), 我们会考虑将它用于各种问题: 采用目光短浅的规则, 一次处理一部分输入, 没有明显的前瞻性。如果一个贪心算法能被证明可以为一个问题的所有实例找到最优解, 那么通常会相当令人惊讶。这样简单的方法可以最优, 基于这一事实, 我们通常会对基本问题的结构有所领悟。

加权区间调度

在区间调度问题中, 我们试图最大化同时可以容纳的请求数量。现在, 更一般地假设每个请求区间 i 具有相关的值, 即权重 (weight), $v_i > 0$; 可以把这想象成, 如果安排第 i 个人的请求就可以赚到的钱。我们的目标是找到最大总值的相容区间子集。

如果对于每个 i , 有 $v_i = 1$, 就是基本的区间调度问题; 但任意值的出现改变了最大化问题的性质。例如, 考虑如果 v_1 超过所有其他 v_i 的总和, 则最优解必须包括区间 1, 而不管整个区间集合的配置如何。因此, 针对此问题的所有算法必须对值非常敏感, 并且当所有值都等于 1 时, 退化为求解 (未加权的) 区间调度的方法。

在存在任意值的情况下, 似乎没有简单的贪心规则能够每次一个地遍历这些区间, 并做出正确的决定。作为替代, 我们采用一种动态规划 (dynamic programming) 技术, 以紧凑的表格方式, 基于所有可能的解求出最佳值, 从而得到非常有效的算法。

二分匹配

当我们考虑稳定匹配问题时, 我们将匹配 (matching) 定义为一组有序的男女配对, 其特征是每位男士和每位女士最多属于一个有序对。然后, 我们将完美匹配 (perfect matching) 定义为一个匹配, 其中每位男士和每位女士都属于某一配对。

我们可以用图来更一般地表达这些概念, 为了做到这一点, 定义二分图 (bipartite graph) 的概念是有用的。我们说图 $G = (V, E)$ 是二分的, 如果它的节点集 V 可以被分割成集合 X 和 Y , 使得每条边在 X 中有一端而在 Y 中有另一端。图 1-5 展示了一个二分图; 通常, 如果想要强调图的“二分性”, 我们将以这种方式绘制它, X 和 Y 中的节点在平行的两列中。但注意, 例如, 图 1-3 中的两个图也是二分图。

现在，在寻找稳定匹配的问题中，匹配是由成对的男女建立的。在二分图的情况下，边是节点对，因此我们说，图 $G=(V,E)$ 中的一个匹配是一组边 $M\subseteq E$ ，其特性是每个节点最多出现在 M 的一条边上。如果每个节点恰好出现在 M 的一条边上，则 M 是完美匹配。

为了理解这确实抓住了在稳定匹配问题中遇到的相同概念，请考虑二分图 G' ，它包含 n 位男士的集合 X ， n 位女士的集合 Y ，一条边从 X 的每个节点到 Y 中的每个节点。于是 G' 中的匹配和完美匹配，正是男士集合与女士集合之间的匹配和完美匹配。

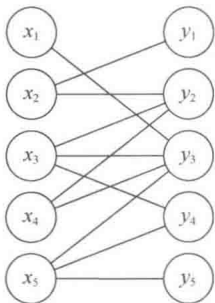


图 1-5 一个二分图

在稳定匹配问题中，我们为此描述添加了偏好。在这里，我们不考虑偏好；但在任意二分图中，问题的性质增加了不同的复杂性来源：从每个 $x\in X$ 到每个 $y\in Y$ ，不一定存在一条边，因此可能的匹配集具有相当复杂的结构。换言之，就好像只有某些男士和女士愿意配对，我们想要弄清楚，如何以与此相符的方式配对许多人。例如，考虑图 1-5 中的二分图 G ： G 中有许多匹配，但只有一个完美匹配。（你看到了吗？）

二分图中的匹配可以模拟将对象分配给其他对象的情况。因此， X 中的节点可以表示作业， Y 中的节点可以表示机器，并且边 (x_i, y_j) 可以表示机器 y_j 能够处理作业 x_i 。然后，完美匹配是一种将每个作业分配给可以处理它的机器的方法，其中每个机器只分配一个作业。在春天，全国各地的计算机科学系经常看到一个二分图，其中 X 是该系教授的集合， Y 是提供的课程集，边 (x_i, y_j) 表示 x_i 教授能够教授课程 y_j 。该图中的完美匹配就是为每位教授分配一门可以教授的课程，以便涵盖每门课程。

因此，二分匹配问题（Bipartite Matching Problem）如下：给定任意二分图 G ，找到最大的匹配。如果 $|X|=|Y|=n$ ，那么当且仅当最大匹配大小为 n 时才存在完美匹配。我们会发现前面讨论的算法技术似乎不足以为这个问题提供有效的算法。然而，有一种非常优雅和有效的算法来找到最大匹配：它归纳地建立越来越大的匹配，在此过程中选择性地回溯。这个过程称为增广（augmentation），它构成了一大类有效可解问题的核心，这些问题称为网络流问题（Network Flow Problem）。

独立集

现在让我们谈谈一个非常一般化的问题，前面大多数问题是它的特例。给定图 $G=(V,E)$ ，如果节点集合 $S\subseteq V$ 中没有两个节点由边连接，我们就说 S 是独立的（independent）。于是，独立集问题（Independent Set Problem）就是：给定 G ，找到尽可能大的独立集。例如，图 1-6 所示的图中独立集的最大大小为 4，由 4 个节点的独立集 $\{1, 4, 5, 6\}$ 实现。

独立集问题描述了以下所有情况：你试图从一组对象中进行选择，而某些对象之间存在成对的冲突（conflict）。假设你有 n 个朋友，而他们中的一些人处不来。如果你不希望任何两人之间关系紧张，那么可以邀请多少朋友参加晚宴？这就是图中最大的独立集，其节点是你的朋友，每个冲突对之间有一条边。

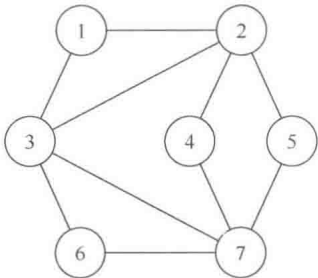


图 1-6 最大独立集的大小为 4 的图

区间调度和二分匹配都可以描述为独立集问题的特殊情况。对于区间调度，定义图 $G=(V,E)$ ，其中节点是区间，每对重叠区间之间有一条边；然后， G 中的独立集合就是区间的相容子集。将二分匹配描述为独立集的一个特例，是一个有点棘手的问题。给定二分图 $G'=(V',E')$ ，被选择的对象是边，拥有共同端点的两条边之间产生冲突。（实际上，这些是不能属于同一个匹

配的两条边。)因此,我们定义了一个图 $G = (V, E)$, 其中节点集 V 等于 G' 的边集 E' 。我们在 V 中的每对元素之间定义了一个边, 这对元素对应于 G' 中有共同端点的两条边。我们现在可以验证, G 的独立集恰好是 G' 的匹配。虽然验证这一点并不复杂, 但需要注意处理这种类型的“边到节点, 节点到边”的转换^①。

鉴于独立集问题的一般性, 解决它的有效算法就会令人印象非常深刻。它必须隐含地包括区间调度、二分匹配, 以及许多其他自然优化问题的算法。

独立集的当前状态是这样的: 没有针对该问题的已知有效算法, 据推测不存在这样的算法。明显的蛮力算法将尝试节点的所有子集, 检查每个子集以查看它是否独立, 然后记录遇到的最大的一个。有可能这接近我们在这个问题上的最优策略。在本书后面会看到, 独立集是一类被称为 NP 完全的 (NP-complete) 问题之一。其中任何一种都没有已知的有效算法, 但从某种意义上说, 它们都是等价的, 即准确地说, 对任何一个问题的解, 都意味着所有这些问题的解。

你自然会问: 关于独立集问题的复杂性, 还有什么好消息吗? 积极的一方面是: 如果一个图 G 包含 1 000 个节点, 我们想让你相信, 它包含一个大小为 100 的独立集 S , 那么这很容易。我们只要向你展示图 G , 用红色圈出 S 的节点, 并让你检查它们中没有两个是由边连接的。因此, 检查某个集合是一个大的独立集, 与实际找到一个大的独立集, 它们的难度差异似乎确实很大。这可能看起来像一个非常基本的观察结果 (它确实是), 但它对于理解这类问题至关重要。此外, 正如我们接下来看到的那样, 问题可能如此困难, 以致在这层意义上, 甚至没有简单的方法来“检查”解。

竞争设施位置

最后, 我们来看第五个问题, 它基于以下双人博弈。考虑两家在全国范围内经营咖啡馆特许经营的大公司 (我们分别称其为 JavaPlanet 和 Queequeg's Coffee), 它们目前正在某个地区竞争市场份额。先是 JavaPlanet 开了一家加盟店; 然后 Queequeg's Coffee 开了一家加盟店; 接着是 JavaPlanet; 接着是 Queequeg's; 如此这般继续下去。假设它们必须遵守分区法规, 这些法规要求两个加盟店不能太靠近, 并且每一方都想让它们的位置尽可能便利。谁会赢?

让我们使这个“博弈”的规则更加具体。问题中的地区分为 n 个区域, 标记为 $1, 2, \dots, n$ 。每个区域 i 都有一个值 b_i , 这是两家公司在那里开设加盟店所获得的收入。最后, 某些区域 (i, j) 是相邻的 (adjacent), 且当地分区法规禁止两个相邻区域各包含一个加盟店, 无论哪个公司拥有它们。(法规也禁止在同一区域开两个加盟店。)我们通过图 $G = (V, E)$ 对这些冲突进行建模, 其中 V 是区域集合, 如果区域 i 和 j 是相邻的, (i, j) 就是 E 中的边。于是, 分区要求是说, 开的所有加盟店必须在 G 中形成一个独立集。

因此, 我们的博弈由两个玩家 P_1 和 P_2 组成, 交替选择 G 中的节点, P_1 先走。在任何时候, 所有选定节点的集合必须在 G 中形成一个独立集。假设玩家 P_2 有一个目标界 B , 我们想知道: P_2 是否有策略, 使得无论 P_1 怎么玩, P_2 都能够选择一组节点, 使其总值至少为 B ? 我们称之为竞争设施位置问题 (Competitive Facility Location Problem) 的一个实例。

例如, 考虑图 1-7 中所示的实例, 并假设 P_2 的目标界是 $B = 20$ 。那么, P_2 确实有获胜策略。然而, 如果 $B = 25$, P_2 就没办法了。

① 对于那些好奇的读者, 我们要指出, 并非每个独立集问题的实例都可以通过这种方式从区间调度或二分匹配中产生; 整个独立集问题确实更一般化。图 1-3a 中的图不能作为区间调度实例中的“冲突图”出现, 图 1-3b 中的图也不能作为二分匹配实例中的“冲突图”出现。

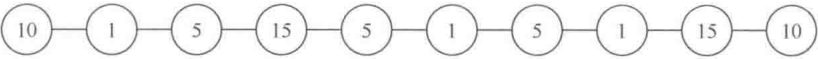


图 1-7 竞争设施位置问题的一个实例

看一下这张图就能想明白，但需要这样的一些情况检查，“如果 P_1 走这里，那么 P_2 会走那里；但如果 P_1 走那里，那么 P_2 会走这里……”这似乎是该问题所固有的：不但在计算上难以确定 P_2 是否有获胜策略，而且在一个适当大小的图上，我们甚至很难说服你 P_2 有一个获胜策略。似乎没有简短的证明。实际上，我们必须向你展示一系列可能的动作，进行冗长的各种情况分析。

这与独立集问题形成对比，在那里，我们认为找到一个大型解很难，但检查一个提出的大型解很容易。这种对比可以形式化为一类 PSPACE 完全问题（PSPACE-complete problem），其中竞争设施位置就是一个例子。严格来说，人们认为 PSPACE 完全问题比 NP 完全问题更难，而且人们推测其解缺乏简短的“证明”，这也是更难的一个标志。事实表明，PSPACE 完全性的概念抓住了大量涉及博弈和规划的问题，其中许多是人工智能领域的基本问题。

带解答的练习

带解答的练习 1

考虑一个小镇，有 n 位男士和 n 位女士希望彼此结婚。每位男士都有一个偏好列表，对所有女士进行排名；每位女士也都有一个偏好列表，对所有男士排名。

所有 $2n$ 个人的集合分为两类：好人和坏人。假设对于某个数 k ， $1 \leq k \leq n - 1$ ，有 k 位好男士和 k 位好女士；因此，有 $n - k$ 位坏男士和 $n - k$ 位坏女士。

每个人都宁愿嫁给任何好人，而不是任何坏人。在形式上，每个偏好列表都具有这样的属性，即对异性的每个好人排名高于异性的每个坏人：它的前 k 个条目是某种顺序的好人（异性），接下来 $n - k$ 个条目是某种顺序的坏人（异性）。

证明在每一个稳定匹配中，每个好男士都与一位好女士结婚。

解 开考这个问题的一种自然方式是假设该断言为假，并尝试推出矛盾。断言为假是什么意思？就是存在某个稳定匹配 M ，其中一位好男士 m 与一个坏女士 w 结婚。

现在，让我们考虑一下 M 中的其他配对看起来是什么样的。有 k 位好男士和 k 位好女士。在这个匹配的 M 中，每位好女士都可以和一位好男士结婚吗？不是：其中一位好男士（即 m ）已经和一个坏女士结婚了，只留下了另外 $k - 1$ 位好男士。因此，即使他们都与好女士结婚，仍然会留下某位好女士与坏男士结婚。

令 w' 是这样一位好女士，她与一个坏男士结婚。现在很容易识别 M 中的不稳定因素：考虑配对 (m, w') 。每个人都很好，但与一个坏伴侣结婚。因此， m 和 w' 中都更喜欢对方，而不是当前的伴侣，因此 (m, w') 是不稳定因素。这与我们关于 M 是稳定的假设相矛盾，因此证明完成。

带解答的练习 2

我们可以考虑稳定匹配问题的一种推广，其中某些男女配对是明确禁止的。在雇主和申请人的例子中，我们可以想象某些申请人只是缺乏必要的资格或证书，因此他们不能在某些公司工作，无论他们看起来多么可取。类似地，在男女之间的婚姻中，我们有 n 位男士的集合 M ， n 位女士的集合 W ，以及配对的集合 $F \subseteq M \times W$ ，他们根本不允许结婚。每位男士 m 会对所有女士 w 进行排名，其中 $(m, w) \notin F$ ；每位女士 w' 会对所有男士 m' 排名，其中 $(m', w') \notin F$ 。

在这个更一般的情况中，我们说匹配 S 是稳定的，如果它没有表现出以下任何类型的不稳定因素。

(i) S 中存在两对，即 (m, w) 和 (m', w') ，其特性是 $(m, w') \notin F$ ， m 更喜欢 w' 而不是 w ， w' 更喜欢 m 而不是 m' 。（通常类型的不稳定因素。）

(ii) 存在一对 $(m, w) \in S$ ，以及一位男士 m' ，使得 m' 不是匹配中任何一对的一部分， $(m', w) \notin F$ ，并且 w 更喜欢 m' 而不是 m 。（更喜欢一个单身男士，而且没被禁止。）

(iii) 存在一对 $(m, w) \in S$ ，和一位女士 w' ，使得 w' 不是匹配中任何一对的一部分， $(m, w') \notin F$ ，并且 m 更喜欢 w' 而不是 w 。（更喜欢一个单身女士，而且没被禁止。）

(iv) 存在一位男士 m 和一位女士 w ，他们都不是匹配中任何一对的一部分，因此 $(m, w) \notin F$ 。（存在两个单身的人，没有什么阻止他们相互结婚。）

注意，在这些更一般的定义下，稳定匹配不一定是完美匹配。

现在我们可以问：对于每组偏好列表和每组禁止配对，是否总是存在稳定匹配？通过执行以下两项操作之一来解决此问题：(a) 给出一种算法，对于任何一组偏好列表和禁止配对，都会产生稳定的匹配；(b) 举例说明一组没有稳定匹配的偏好列表和禁止配对。

解 面对稳定匹配问题的变化，Gale-Shapley 算法极为健壮。因此，如果你遇到问题的新变化，又找不到稳定匹配的反例，那么检查直接调整 G-S 算法是否事实上会产生稳定匹配，这通常是个好主意。

事实证明就是这种情况。我们将证明，即使在这种具有禁用对的更一般的模型中，也总是存在稳定匹配，我们将通过调整 G-S 算法来做到。为此，我们考虑一下为什么不能直接使用初始的 G-S 算法。当然，困难在于 G-S 算法根本不知道禁止配对，因此，while 循环中的条件

While 存在一位男士 m ，他是自由的，而且还没向任何一位女士求过婚

不再有效：我们不希望 m 向一位女士 w 求婚，而 (m, w) 是被禁止的。

因此，我们考虑修改 G-S 算法，只做一点改变：将 while 循环修改为

While 存在一位男士 m ，他是自由的，而且还没向任何一位女士 w 求过婚，其中 $(m, w) \notin F$

下面是完整的算法。

最初所有 $m \in M$ 和 $w \in W$ 都是自由的

While 存在一位男士 m ，他是自由的，而且还没向任何一位女士 w 求过婚，其中 $(m, w) \notin F$

 选择这样一位男士 m

 设 w 是 m 的偏好列表中排名最高的女士，而 m 还没有向她求过婚

 If w 是自由的 then

(m, w) 变成订婚状态

 Else w 当前与 m' 订婚

 If w 更喜欢 m' 而不是 m then

m 保持自由

 Else w 更喜欢 m 而不是 m'

(m, w) 变成订婚状态

m' 变成自由

 Endif

 Endif

Endwhile

Return 订婚配对的集合 S

我们现在证明，根据我们对稳定的新定义，这会产生稳定匹配。

首先，本书中的事实 (1.1)、(1.2) 和 (1.3) 保持为真（特别是，算法将在最多 n^2 次迭代后终止）。此外，我们不必确认得到的匹配 S 是完美的（事实上，它可能不是）。我们还注意到另外一对事实。如果 m 是一位男士，且不属于 S 中的一对，那么 m 必须向每个无禁止的女士求婚；如果 w 是一位女士，且不属于 S 中的一对，那么一定没有男士向 w 求过婚。

最后，我们只需要证明：

(1.9) 返回的匹配 S 没有不稳定因素。

证 我们对不稳定因素的一般定义有 4 个部分：这意味着我们必须确保 4 种不良事件都不会发生。

首先，假设存在类型 (i) 的不稳定因素，由 S 中的 (m, w) 和 (m', w') 对构成，其性质为 $(m, w') \notin F$ ， m 更喜欢 w' 而不是 w ， w' 更喜欢 m 而不是 m' 。因此， m 肯定向 w' 求过婚；所以 w' 拒绝过 m ，她更喜欢她的最终伴侣而不是 m' ——矛盾。

其次，假设存在类型 (ii) 的不稳定因素，由一对 $(m, w) \in S$ 和一位男士 m' 构成，因此 m' 不是匹配中任何一对的一部分， $(m', w) \notin F$ ， w 更喜欢 m' 而不是 m 。那么 m' 肯定向 w 求过婚并被拒绝；同样，又推出 w 更喜欢她的最终伴侣而不是 m' ——矛盾。

再次，假设存在类型 (iii) 的不稳定因素，由一对 $(m, w) \in S$ 和一位女士 w' 构成，因此 w' 不是匹配中任何一对的一部分， $(m, w') \notin F$ ， m 更喜欢 w' 而不是 w 。那么根本没有人向 w' 求过婚；特别是， m 从来没向 w' 求过婚，因此他肯定更喜欢 w 而不是 w' ——矛盾。

最后，假设存在类型 (iv) 的不稳定因素，由男士 m 和女士 w 构成，这两者都不是匹配中任何一对的一部分，使得 $(m, w) \notin F$ 。但是， m 要成为单身，他肯定向每一个无禁止的女士求过婚；特别是，他肯定向 w 求过婚，这意味着她将不再是单身——矛盾。■

练习

1. 确定以下陈述是真是假。如果是真，请简要解释。如果是假，请给出一个反例。

真或假？在稳定匹配问题的每个实例中，存在包含配对 (m, w) 的稳定匹配，使得 m 在 w 的偏好列表上排名第一，且 w 在 m 的偏好列表上排名第一。

2. 确定以下陈述是真是假。如果是真，请简要解释。如果是假，请给出一个反例。

真或假？考虑稳定匹配问题的实例，其中存在男士 m 和女士 w ，使得 m 在 w 的偏好列表上排名第一，且 w 在 m 的偏好列表上排名第一。那么，在该实例的每个稳定匹配 S 中， (m, w) 对属于 S 。

3. 还有许多其他场景，我们可以提出与某种“稳定”原则相关的问题。下面一个涉及两个企业之间的竞争。

假设有两个电视网络，我们称之为 A 和 B 。有 n 个黄金时间的节目安排时段，每个网络有 n 个电视节目。每个网络都想制定一个时间表（将每个节目分配到不同的时段），以便尽可能多地吸引市场份额。

以下是我们根据他们的时间表，确定两个网络相对表现好坏的方式。每个节目都有固定的收视率，这取决于去年观看的人数；我们假设没有两个节目具有完全相同的收视率。与其他网络在该时段安排的节目相比，如果某网络在该时段安排的节目具有更高的收视率，则该网络赢得该时段。每个网络的目标是赢得尽可能多的时段。

假设在秋季的第一周, 网络 A 公布了时间表 S , 网络 B 公布了时间表 T 。根据这一对时间表, 基于上述规则, 每个网络赢得了某些时间段。如果两个网络都不能单方面改变自己的时间表并赢得更多时段, 我们就说这一对时间表 (S, T) 是稳定的。也就是说, 不存在时间表 S' , 使得网络 A 在 (S', T) 的情况下赢更多时段, 超过 (S, T) 的情况; 同样, 不存在时间表 T' , 使得网络 B 在 (S, T') 的情况下赢更多时段, 超过 (S, T) 的情况。

Gale 和 Shapley 的问题对这种稳定性的类比如下: 对于每组电视节目和收视率, 是否总有一对稳定的时间表? 通过执行以下两项操作之一来解决此问题:

- (a) 给出一种算法, 对于任何一组电视节目和相关的收视率, 产生一对稳定时间表;
- (b) 举例说明一组没有稳定时间表的电视节目和相关收视率。

4. Gale 和 Shapley 于 1962 年发表了关于稳定匹配问题的论文; 但是, 为了解决将医生分配到医院的问题, 他们的算法的一个版本已经由美国国家医生匹配计划使用了 10 年。

基本上, 情况如下。有 m 家医院, 每家医院都有一定数量的空缺职位来招聘医生。某一年, 有 n 个医学院学生毕业, 每个都有兴趣加入其中一家医院。每个医院按照偏好顺序对学生进行排名, 每个学生按偏好顺序排列医院。我们假设有较多的学生毕业, 超过 m 家医院的空缺职位。

当然, 我们的兴趣在于找到一种方法, 将每个学生分配到最多一家医院, 这样就可以填补所有医院的所有空缺职位。(由于我们假设学生过剩, 因此会有一些学生没有被分配到任何医院。)

如果没有出现以下两种情况之一, 我们就说, 学生到医院的分配是稳定的。

- 第一类不稳定因素: 存在学生 s 和 s' , 以及医院 h , 使得
 - ◆ s 被分配给 h , 且
 - ◆ s' 没有被分配到任何医院, 且
 - ◆ h 更喜欢 s' 而不是 s 。
- 第二类不稳定因素: 存在学生 s 和 s' , 以及医院 h 和 h' , 使得
 - ◆ s 被分配给 h , 且
 - ◆ s' 被分配给 h' , 且
 - ◆ h 更喜欢 s' 而不是 s , 且
 - ◆ s' 更喜欢 h 而不是 h' 。

这样我们基本上得到了稳定匹配问题, 除了 (i) 医院通常需要一个以上的医生, 以及 (ii) 学医的学生有过剩。

证明总是存在一个学生到医院的稳定分配, 并给出一个算法, 找到一个稳定分配。

5. 如本书中讨论的, 稳定匹配问题假定所有男士和女士都有全序的偏好列表。在这个问题中, 我们将考虑该问题的一个版本, 其中男士和女士可以在某些选择中无所谓。与以前一样, 我们有 n 位男士的集合 M 和 n 位女士的集合 W 。假设每位男士和每位女士都对异性排名, 但现在我们允许排名中的并列关系。例如 ($n=4$), 一位女士可以说 m_1 排在第一名; 第二名是 m_2 和 m_3 并列 (她对她们没有偏好); m_4 排在最后。我们会说 w 更喜欢 m 而不是 m' , 如果在她的偏好列表上, m 的排名高于 m' (它们没有并列)。

由于排名方式无所谓, 因此稳定性可能存在两种自然的概念。对于每一种, 我们可以问是否存在稳定匹配, 如下所述。

(a) 强不稳定因素存在于完美匹配 S 中, 由男士 m 和女士 w 构成, 使得 m 和 w 中的每一个都更喜欢另一个人, 而不是他们在 S 中的伴侣。是否总是存在没有强不稳定因素的完美匹配? 要么给出一组具有偏好列表的男士和女士的例子, 其中每个完美匹配都具有强不稳定因素; 要么给出一个算法, 确保找到没有强不稳定因素的完美匹配。

(b) 弱不稳定因素存在于完美匹配 S 中，由男士 m 和女士 w 构成，使得他们在 S 的伴侣分别是 w' 和 m' ，且以下之一成立：

- m 更喜欢 w 而不是 w' ，且 w 要么更喜欢 m 而不是 m' ，要么对这两种选择无所谓；
- w 喜欢 m 而不是 m' ，且 m 要么更喜欢 w 而不是 w' ，要么对这两种选择无所谓。

换言之， m 和 w 之间的配对要么两人都喜欢对方，要么一个喜欢对方而另一个无所谓。是否总是存在没有弱不稳定因素的完美匹配？要么给出一组具有偏好列表的男士和女士的例子，其中每个完美匹配都有弱不稳定因素；要么给出一个算法，确保找到没有弱不稳定因素的完美匹配。

6. PSL 是一家航运公司，拥有 n 艘船，并为 n 个港口提供服务。它的每艘船都有一个时间表，表示每个月的每一天，它当时正在哪个港口，或者正在海上。（你可以假设这里的“月”有 m 天， $m > n$ 。）每艘船每个月都会访问每个港口一天。出于安全原因，PSL 公司有以下严格要求：

(†) 同一天不能有两艘船在同一个港口。

该公司希望通过以下方案在本月对所有船舶进行维护。他们想截断每艘船的时间表：对于每艘船 S_i ，有一天它会到达预定的港口，并在那个月的剩余时间里留在那里维护。这意味着 S_i 不会访问那个月的时间表中其余的港口（如果有的话），但这没关系。因此， S_i 的时间表截断就是包含原来的时间表，直到某个特定日期，它到达港口 P ；截断时间表的其余部分就是让它停留在港口 P 中。

现在公司的问题如下：给定每艘船的时间表，找出每艘船的截断方案，以便条件 (†) 继续保持：同一天不能有两艘船在同一个港口。

证明总可以找到这样的一组截断，并给出一个算法找到它们。

例 假设有两艘船和两个港口，这个“月”有 4 天。假设第一艘船的时间表是

港口 P_1 ；在海上；港口 P_2 ；在海上

而第二艘船的时间表是

在海上；港口 P_1 ；在海上；港口 P_2

然后选择截断的（唯一）方法，是让第一艘船从第三天开始留在港口 P_2 ，并让第二艘船从第二天开始留在港口 P_1 。

7. 你的一些朋友正在为 CluNet 工作，该公司是大型通信网络的建造商，他们正在研究算法，用于切换特定类型的输入/输出交叉开关。

下面是基本情况。有 n 条输入线和 n 条输出线，每条线都从一个源点指向一个终点。在一个名为接线盒的特殊硬件中，每条输入线与每条输出线相接于一个不同的点上。线上的点按照从源点到终点的方向自然排序；对于同一条线上的两个不同点 x 和 y ，如果 x 比 y 更接近源，我们就说 x 在 y 的上游，否则我们就说 x 在 y 的下游。一条输入线与输出线相交的顺序不一定与另一条输入线与输出线相交的顺序相同。（输出线与输入线相遇的顺序也类似。）图 1-8 给出了这样一组输入线和输出线的例子。

接下来是这种情况的开关部件。每条输入线都传送一个不同的数据流，这个数据流必须切换到其中一条输出线上。如果在接线盒 B 处，输入流 i 被切换到输出 j ，则该流通过输入 i 上 B 上游的所有接线盒，然后通过 B ，然后通过输出 j 上 B 下游的所有接线盒。哪个输入数据流切换到哪个输出线无关紧要，但每个输入数据流必须切换到不同的输出线。此外（这是棘手的约束），在切换操作之后，没有两个数据流可以通过相同的接线盒。

最后是问题。证明对于输入线和输出线彼此相交的任何指定模式（每对只相交一次），总可以找到数据流的有效切换——其中每个输入数据流切换到不同的输出，且在产生的流中没有两个通过相同的接线盒。此外，给出一种算法来找到这种有效切换。

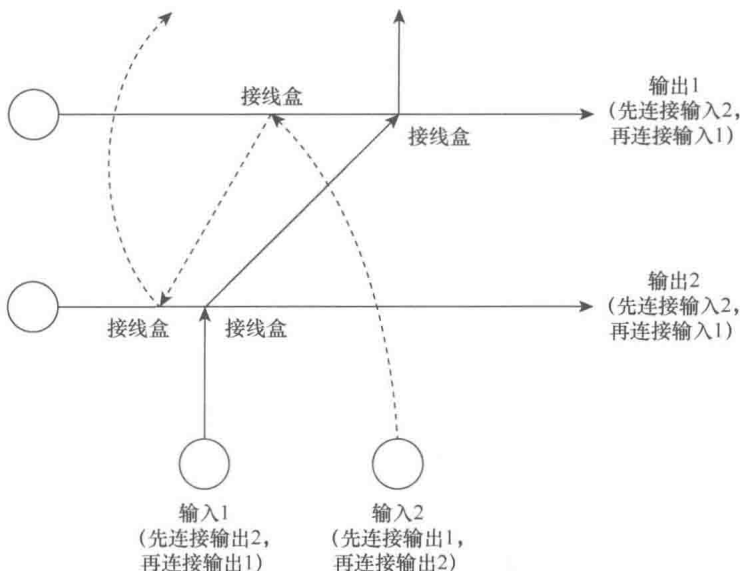


图 1-8 包含两根输入线和两根输出线的示例。输入 1 与输出 2 的连接点，在它 与输出 1 的连接点上游；输入 2 与输出 1 的连接点，在它 与输出 2 的连接点上游。有效解是将输入 1 的数据流切换到输出 2，将输入 2 的数据流切换到输出 1。然而，如果输入 1 的流切换到输出 1，输入 2 的流切换到输出 2，那么两个流都会通过输入 1 和输出 2 连接处的接线盒——这是不允许的

8. 在这个问题中，我们将探讨稳定匹配问题中的真实性问题，特别是 Gale-Shapley 算法中的真实性问题。基本问题是：男士或女士是否能在自己的偏好上撒谎，从而得到更好的结果？更具体地说，我们假设每个参与者都有一个真正的偏好顺序。现在考虑一位女士 w 。假设 w 更喜欢男士 m 而不是 m' ，但是 m 和 m' 在她的偏好列表中都很低。是否有这种情况，通过交换 m 和 m' 在她的偏好列表中的顺序（即假装声称她更喜欢 m' 而不是 m ），并用这个假偏好列表运行算法， w 最终会得到一个她更喜欢的男士 m'' ，超过 m 和 m' ？（我们可以对男士提出同样的问题，但在这个问题中，我们关注女士的情况。）

通过执行以下两项操作之一来解决此问题：

(a) 证明：对于任何一组偏好列表，在列表中切换一对的顺序不能改善 Gale-Shapley 算法中女士的伴侣；

(b) 给出一组偏好列表的例子，其中存在一次交换，可以改善交换偏好的女士的伴侣。

注释和进一步阅读

稳定匹配问题首先由 Gale 和 Shapley（1962）定义和分析；根据 David Gale 的说法，他们对这个问题的动机来自于他们当时在《纽约客》上读到的有关大学入学程序错综复杂的故事（Gale, 2001）。稳定的匹配本身已经成为一个研究领域，在 Gusfield 和 Irving（1989）以及 Knuth（1997c）的著作中有介绍。Gusfield 和 Irving 也对稳定匹配问题的“平行”历史提供了一个很好的调查：它作为一种技术发明，在医疗和其他行业中用于匹配申请人和雇主。

正如本章所讨论的，我们的 5 个典型问题分别是本书要讨论的贪心算法、动态规划、网络流、NP 完全性和 PSPACE 完全性的核心问题。我们将在本书的后面讨论这些问题。